

## COMPUTATIONAL THINKING USING PYTHON

Global Solution 2026/1 | 1TDSPH | FIAP

---

# OrbitSafe

---

*Plataforma de Monitoramento Emergencial via Satélites*

### Integrantes do Grupo

**Eduardo Damasio Guelere** | RM: 569960

**Milena Silva Conegin** | RM: 568923

**Isabelle Ferreira Neri Feitoza** | RM: 573507

**Andre Luiz Ramos Forastieri** | RM: 572203

**Marina Fernandes Gomes Mesquita** | RM: 571265

São Paulo, 2026

# 1. Descrição da Solução Proposta

O OrbitSafe é uma plataforma de monitoramento de emergências que utiliza tecnologia de satélites para manter a comunicação em regiões remotas e em situações de desastre. O sistema foi desenvolvido como parte do projeto da Global Solution 2026 da FIAP, com o tema O Espaço e a Nova Fronteira, conectando a exploração espacial com problemas reais enfrentados aqui na Terra.

O problema central abordado é a perda de conectividade durante desastres naturais, como enchentes, queimadas, deslizamentos e tempestades. Quando a infraestrutura tradicional falha, incluindo torres de sinal, redes terrestres e energia elétrica, cidades inteiras podem ficar isoladas, dificultando o acionamento de equipes de resgate e colocando vidas em risco.

Para enfrentar esse desafio, o OrbitSafe permite o cadastro de usuários afetados por desastres, além do registro de emergências com informações como tipo, localização e grau de severidade. A plataforma também realiza consultas de risco climático com base em dados de temperatura, chuva e velocidade do vento, auxiliando na prevenção e resposta rápida a eventos críticos.

Utilizando Python, o sistema automatiza o monitoramento climático, simula a ativação de satélites de comunicação em regiões sem conectividade, monitora focos de queimadas em diferentes biomas brasileiros e gera relatórios completos contendo usuários cadastrados e ocorrências registradas, fornecendo informações estratégicas para apoiar a tomada de decisões em situações de emergência.

## 2. Conceitos Técnicos Envolvidos

O código do OrbitSafe aplica os principais conceitos da disciplina, conforme descrito a seguir:

### 2.1 Decisão e Seleção (if, elif, else e match-case)

Estruturas condicionais são utilizadas em diversas funções do sistema. Na função `registrar_emergencia()`, a instrução `if/elif/else` determina qual equipe de atendimento será acionada conforme o grau de severidade da ocorrência (Crítico, Alto, Médio ou Baixo). Da mesma forma, em `consultar_conectividade()`, o sistema avalia se a internet está disponível e, caso contrário, aciona a comunicação via satélite. A função `calcular_risco()` também emprega condicionais para classificar o nível de risco climático em Alto, Médio ou Baixo, de acordo com os valores de temperatura, chuva e vento. Utilização do `match-case` no `exibir_menu()` apresentando ao usuário todas as opções disponíveis no sistema OrbitSafe.

### 2.2 Estruturas de Repetição (while e for)

O loop `while True` no programa principal mantém o menu ativo até o usuário escolher sair (opção 0). Loops `while` é utilizado nas validações:

Def `cadastrar_usuario`:

Nome (deve conter apenas letras, sem espaços vazios, sem números ou símbolos)

CPF (deve conter exatamente 11 dígitos numéricos), não podendo ter o campo vazio, repetindo a solicitação até o dado ser válido.

Cidade (deve conter apenas letras)

Estado (O estado não pode tem o campo vazio e ser digitado com siglas, mas com o nome completo, ex: São Paulo).

Def consultar\_conectividade:

Regiao (Não aceita espaços vazios e números)

Status (Obriga o usuário a responder “sim” ou “não”, não aceitando números e nem espaços vazios)

exibir\_menu:

Cidade: (Não aceita espaço vazio e nem números )

O loop for percorre a lista de regiões em monitorar\_queimadas() e, em ver\_relatorios(), itera sobre as listas de usuários e emergências para exibir todos os registros armazenados.

## 2.3 Estruturas de Dados: Listas, Dicionários e Strings

Duas listas globais usuarios e emergencias, armazenam os registros durante a execução. Cada elemento é um dicionário com chaves como nome, cpf, cidade, estado (usuário) e tipo, local, grau (emergência). A lista regioes em monitorar\_queimadas() contém strings com os biomas brasileiros. Os dicionários tipos\_validos e graus\_validos mapeiam as opções do menu a seus respectivos rótulos, garantindo validação eficiente das entradas.

## 2.4 Sub-algoritmos: Funções e Procedimentos (def)

O código está organizado em funções independentes, cada uma responsável por uma funcionalidade específica do sistema. A função calcular\_risco(temperatura, chuva, vento) recebe três parâmetros e retorna uma string com o nível de risco, demonstrando passagem de parâmetros e retorno de valor. A função consultar\_risco\_climatico(cidade) recebe a cidade como parâmetro e chama internamente calcular\_risco(), ilustrando a composição de subalgoritmos. As demais funções (cadastrar\_usuario, registrar\_emergencia, consultar\_conectividade, monitorar\_queimadas, ver\_relatorios) são procedimentos que executam operações diretamente.

## 2.5 Tratamento de Erros e Validação de Dados

O sistema aplica blocos try/except para capturar erros de conversão de tipos (ValueError), como nas entradas numéricas de temperatura, chuva, vento e focos de queimadas. Erros de chave em dicionários (KeyError) são tratados para lidar com opções inválidas no menu. As validações com while garantem que CPF e estado só sejam aceitos no formato correto, melhorando a robustez e a usabilidade da aplicação.

# 3. Funcionalidades do Sistema

O menu principal do OrbitSafe possui 7 opções funcionais, além da opção de saída:

| Opção | Funcionalidade | Descrição |
|-------|----------------|-----------|
|-------|----------------|-----------|

|     |                                |                                                                                               |
|-----|--------------------------------|-----------------------------------------------------------------------------------------------|
| [1] | <b>Sobre o Projeto</b>         | Exibe uma descrição textual da solução OrbitSafe diretamente no terminal.                     |
| [2] | <b>Cadastrar Usuário</b>       | Solicita nome, CPF (validado com 11 dígitos) e localização. Armazena em lista de dicionários. |
| [3] | <b>Registrar Emergência</b>    | Registra tipo, local e grau da ocorrência. Aciona equipes conforme a criticidade.             |
| [4] | <b>Risco Climático</b>         | Recebe dados climáticos da cidade e calcula o nível de risco (Baixo, Médio ou Alto).          |
| [5] | <b>Consultar Conectividade</b> | Verifica o status da internet e, se indisponível, simula ativação do satélite SAT-ALPHA.      |
| [6] | <b>Monitorar Queimadas</b>     | Registra focos de incêndio nos 5 biomas brasileiros e classifica o nível de alerta.           |
| [7] | <b>Ver Relatórios</b>          | Exibe todos os usuários cadastrados e emergências registradas na sessão atual.                |
| [0] | <b>Sair</b>                    | Encerra o programa com mensagem de despedida.                                                 |

## 4. Exemplos de Execução do Sistema

A seguir, prints das principais telas de execução do OrbitSafe via terminal.

### 4.1 Menu Principal

```

=====
OrbitSafe - MENU PRINCIPAL
=====
[1] Sobre o projeto
[2] Cadastrar usuário
[3] Registrar emergência
[4] Consultar risco climático
[5] Consultar conectividade
[6] Monitorar queimadas
[7] Ver relatórios
[0] Sair
=====
Escolha uma opção: 

```

## 4.2 Cadastro de Usuário

```
=====
CADASTRO DE USUÁRIO
=====
Digite seu nome: Eduardo
Digite seu CPF (somente números, 11 dígitos): 12345678910
Digite sua cidade: sbc
Digite seu estado (ex: SP): sp

Usuário cadastrado com sucesso!
Nome : Eduardo
CPF : 12345678910
Cidade: sbc - SP
```

## 4.3 Registro de Emergência

```
=====
REGISTRAR EMERGÊNCIA
=====
Tipos de emergência:
[1] Enchente
[2] Queimada
[3] Deslizamento
[4] Tempestade
Escolha o tipo: 3
Local da ocorrência: sbc
Grau de emergência:
[1] Baixo
[2] Médio
[3] Alto
[4] Crítico
Escolha o grau: 3

Emergência registrada!
Tipo : Deslizamento
Local: sbc
Grau : Alto
>> Acionando: Bombeiros + Defesa Civil
=====
```

## 4.4 Consulta de Risco Climático

```
Digite o nome da cidade: sbc

=====
CONSULTA DE RISCO CLIMÁTICO
=====
Temperatura atual (°C): 30
Volume de chuva (mm): 100
Velocidade do vento (km/h): 20

Resultado para: sbc
Temperatura : 30.0 °C
Chuva : 100.0 mm
Vento : 20.0 km/h
Nível de risco: Alto
=====
```

## 4.5 Consulta de Conectividade

```
=====
CONSULTA DE CONECTIVIDADE
=====
Digite a região ou cidade: sbc
A internet está funcionando? (sim/nao): nao

Conectividade FALHOU em: sbc
Ativando comunicação via satélite...
Satélite SAT-ALPHA acionado para a região.
=====
```

```
=====
CONSULTA DE CONECTIVIDADE
=====
Digite a região ou cidade: sbc
A internet está funcionando? (sim/nao): sim

Conectividade OK em: sbc
Sinal de internet disponível.
=====
```

## 4.6 Monitoramento de Queimadas

```
=====
MONITORAMENTO DE QUEIMADAS
=====
Focos detectados em Amazônia: 20
-> Amazônia : 20 focos - Normal
Focos detectados em Cerrado: 100
-> Cerrado : 100 focos - ATENÇÃO
Focos detectados em Pantanal: 150
-> Pantanal : 150 focos - ATENÇÃO
Focos detectados em Caatinga: 200
-> Caatinga : 200 focos - ATENÇÃO
Focos detectados em Mata Atlântica: 1000
-> Mata Atlântica : 1000 focos - CRÍTICO
=====
```

## 4.7 Relatório do Sistema

```
=====
RELATÓRIOS DO SISTEMA
=====
Usuários cadastrados: 2
Emergências registradas: 2

-- Usuários --
- Eduardo | 12345678910 | sbc - SP
- Carlos | 10987654321 | Diadema - SP

-- Emergências --
- Deslizamento | sbc | Grau: Alto
- Enchente | Diadema | Grau: Baixo
=====
```

## 4.8 Validação de Dados e Tratamento de Erros

```
=====
CADASTRO DE USUÁRIO
=====
Digite seu nome: José
Digite seu CPF (somente números, 11 dígitos): 123
Erro: CPF inválido. Digite apenas os 11 números, sem pontos ou traços.
Digite seu CPF (somente números, 11 dígitos): ola
Erro: CPF inválido. Digite apenas os 11 números, sem pontos ou traços.
Digite seu CPF (somente números, 11 dígitos): 2222222222
Digite sua cidade: sbc
Digite seu estado (ex: SP): São Paulo
Erro: Estado inválido. Digite a sigla com exatamente 2 letras (ex: SP, RJ, MG).
Digite seu estado (ex: SP): SP
```